Digital Design
Combinational logic



Combinational logic: Agenda

- Number systems
- Radix conversions
- K-maps, min-terms, max terms
- Logic gates
- Realization of logic gates using mux's and universal gates
- Compliments (1/2/9/10's complement)
- Arithmetic operations using compliments
- Boolean expression minimization, Dmorgan theorems
- POS and SOP
    - Conversion and realization
- Adders
    - Half adder
    - Full adder
- Subtractor
    - Half subtractor
    - Full subtractor
- Adders and subtractors using Mux's
- Multiplexers
    - Realizing bigger Mux's using smaller Mux's
- Decoders and Encoders
- Decoders and Encoders using Mux and Demux
- Bigger Decoder/Encoder using smaller Decoder/Encoder
- Comparators



Sequential logic: Agenda

- Latch, Flipflop
- Latch, Flipflop using Gates or Mux's
- Different types of FFs
  - Truth table
  - Excitation tables
- Applications of FF's, Latches
  - Counters
  - Shift registers
  - Synchronizers for clock domain crossing
  - FSM's
    - Mealy, Moore FSM
    - Different encoding styles
  - Frequency dividers
  - Frequency multiplication
- STA

Why learn digital design?

VLSIGURU

- Imagine 2 mobile phone SOC chips
  - Both implementing same features with almost same performance
    - 1st chip requires 1 million gates
    - 2nd chip requires 0.8 million gates
  - Which one we prefer?
    - Reduces the cost
    - Reduces the area
    - Reduces the power consumption
    - Makes the design more portable
    - Design complexity also reduces
  - What is the role of Digital and Analog design?
    - These are the various techniques which help us implement the required logic using minimal number of gates or analog elements, **while not impacting the design functionality**.
  - Digital design
    - k-maps, POS, SOP, D-Morgan
- What is the role of Digital design engineer?
  - Digital design is supposed to come up with the digital circuit for a given design.

Number systems

- Binary, Decimal, Octal, Hexadecimal
  - There can by any base system from 2 to 64
- How we write a number in various radix formats
  - size, radix, value
  - <size>'radix<value>
  - 32'h1234_ABCD
    - 32: size
    - h : radix
    - 1234_ABCD : value
- Represent 350 with size of 12 using all radix
- Radix conversions
  - Decimal to octal, hexadecimal, binary
- Representing floating numbers using binary, octal, decimal

Vector value assignment

VLSIGURU

Vector data Value assignment : <size>'<radix><value>

- reg [31:0] data = 32'h1F2F_3F4F; //vector
  - 10101001_0101_0101_0101_0010_1010 underscore is used for readability.
- reg valid; //scalar
- radix : Binary (b or B), Octal (o or O), Decimal (d or D), Hexadecimal (h or H)
- Value: Can be written using number as per radix along with '_'
- reg [31:0] addr;
- how to assign value to addr?
  - **3 things**
    - size : 32 bits
    - radix : b/o/d/h
    - value : user needs to specify (ex: 200 in decimal)
  - addr = <size>'Radix<value>;
  - addr = 32'd200; //Decimal format
  - how to give value in hexadecimal?
    - 200 = 128 + 64 + 8
    - addr = 32'b1100_1000 = 32'hC8
  - how to give value in Octal?
  - addr = 32'b11_001_000 = 32'o310
- what if size of addr is 16 bits?
  - addr = 16'o310;

Radix conversion

VLSIGURU

Ex:

8'hax same as 8'b1010_xxxx

12'o3zx7 same as 12'h011_zzz_xxx_111

1 character in octal means 3 characters in binary

10'd10 is same as 10'b00_0000_1010 (same as 10'b1010)

Assignment: (varname = wdata)

Value = 350 decimal

Size = 15

Represent above in all 4 radix formats.

- wdata = 15'd350;
- Wdata = 15'h15e; //1*256 + 5*16+14 = 256+80+14 = 350
- How to convert this to Octal?
  - First convert to Binary
  - Wdata = 15'b0001_0101_1110;
  - Group them as 3 bits each
  - Wdata = 15'b000_101_011_110;
- Wdata = 15'o0536

Hexa to Binary conversion mapping

VLSIGURU

Hexa	Binary
x	xxxx (note: x is small in Hexa)
z	zzzz
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111
X	some bits are x (00x1, xx00, not all are x)
Z	some bits are z (zz01, zz00, not all are z)

In competetion of X and Z, X always wins.

Octal	Binary
x	xxx
z	zzz
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111
X	some bits of x

Decimal to any other base conversion

- To get integer part in required base
  - take successive divisions of decimal integer part with required base and accumulate **remainders in reverse order (bottom up order)**
- To get fraction part in required base
  - take successive multiplication of decimal fraction part with required base and accumulate **integers in original order(top-down order)**.
  - During this, some patterns can repeat, in those case keep the first repetition pattern only.

Decimal to octal conversion

VLSIGURU

Example 1

Convert 120 (decimal) to its equivalent octal number.

	Octal number	*Remainder*
8	120	
8	15	0
8	1	7
	0	1

∴ the octal equivalent of 120 is 170.

Example 2

Convert 460 (decimal) to its equivalent octal number.

	Octal number	*Remainder*
8	460	
8	57	4
8	7	1
	0	7

∴ the octal number = 714.

$0.513 \times 8 = 4.104$ 4

$0.104 \times 8 = 0.832$ 0

$0.832 \times 8 = 6.656$ 6

$0.656 \times 8 = 5.248$ 5

$0.248 \times 8 = 1.984$ 1

$(0.513)_{10} = (0.40651...)_8$

Complete answer is $(152.512)_{10} = (230.40651...)_8$

converting decimal to hexadecimal conversion

0.565 x 16 = **9**.04	9
0.04 x 16 = **0**.64	0
0.64 x 16 = **10**.24	10 = A
0.24 x 16 = **3**.84	3
0.84 x 16 = **13**.44	13 = D
0.44 x 16 = **7**.04	7
0.04 x 16 = **0**.64	0

$(0.565)_{10} = (0.90A3D70...)_{16}$

Complete answer is $(2607.565)_{10} = (A2F.\ 90A3D70...)_{16}$

Decimal to hexadecimal

- The conversion method of decimal to hexadecimal is the same as that of decimal to binary except that the base taken is 16 instead of 2.
- For example, to convert 765.245_{10} to the hexadecimal equivalent, do the following:

Integer Part

16	765
16	47 - 13
16	2 - 15
	0 - 2

Fractional Part

0.245
x 16
3.920
x 16
14.720
x 16
11.520

$765.245_{10} = 2FD.3EB_{16}$

VLSIGURU

2-Any base r-to-Decimal Conversion

- Base-r to decimal: multiply digits with their corresponding weights.
- **Examples**
  - $(1101.101)_2 = 1\times2^3 + 1\times2^2 + 1\times2^0 + 1\times2^{-1} + 1\times2^{-3}$
    $= 8 + 4 + 1 + 0.5 + 0.125$
    $= (13.625)_{10}$
  - $(572.6)_8 = 5\times8^2 + 7\times8^1 + 2\times8^0 + 6\times8^{-1}$
    $= 320 + 56 + 2 + 0.75 = (378.75)_{10}$
  - Convert $3b2_{16}$ to decimal.
    $N = 3*16^2 + 11*16^1 + 2*16^0 = 946_{10}$
  - Convert 370_8 to decimal.
    $N = 3*8^2 + 7*8^1 + 0*8^0 = 192 + 56 + 0 = 248_{10}$

Radix conversions

- Representing a number in different radix
  - 135 in different radix using 10 bits
  - -135 in different radix using 10 bits
- How to radix conversions
  - Decimal -> Binary, Octal, Hexa
    - **Integer part**: Divide and take all quotient from bottom to top
    - **Fractional part**: should be multiplied and take all quotient from top to bottom
  - Octal to hexa, hexa to octal
    - Convert to binary and regroups the bits, then binary to other format.
  - Octal, hexa, binary to decimal
    - Multiple with r**N and sum all of them
      - r : radix
      - N: number position

Radix conversions ...

- Decimal -> Binary, Octal, Hexa
  - (177.57)10 in different formats

Radix calculations

- Q#1 www.vlsiguru.com/digital-design-complete/

Number of bits required

- For a signed variable of n bits size, value range
- -2**n to 2**n-1
  - -8 to 7 : 4 bits
  - -55 : 7 bits required
    - +ve number, add 0 in the beginning and then take complement.
  - -129

Questions

- convert (25.05) to binary
- convert (25.05) to hexadecimal
- convert (25.05) to octal
- convert 319 to binary
- convert 1147 to octal
- convert (6051.703)8 to decimal
- convert(C09.2F0)16 to binary
- convert (5037.621)8 represent it in hexadecimal

1/2/9/10's complement

- -25
  - 00011001
  - 1s complement : 11100110
  - 2s complement : 11100111
- sign extension
  - MSB bits extended as 1's or 0's matching the sign bit
- 2's complement = 1's complement + 1
- 10's complement = 9's complement + 1
- X' complement : X-1's complement and add '1'

- Minimum of bits required to represent a number
  - 5 bits can represent : -16 to 15
  - 6 bits can represent: -32 to 31
- why we prefer using 2's complement?
  - 1's complement, 0 has 2 different values
    - +0 using 5 bits: 00000
    - -0 using 5 bits: 11111
  - 2's complement, 0 has only 1 value
    - +0 & -0 are both 00000

1/2/9/10's complement ...

- + number is same in all representations (we don't take complement)
- - number is different in all representations
- Need so many radix?
    - Various radix formats for user convenience
    - ASCI convention is in Hexa
    - Hardware in turn converts it to Binary.
- Why we need so many compliments?
    - Is it used in real hardware processing?
- For binary number systems
    - Use 1's and 2's complement
        - Addition and subtraction
    - Signed number can be represented using compliment
- Complementary of any number(X) is the negative number(-X)
    - It can be any complementary
- 2's complement will require lesser hardware.
    - Similarly for 10's complement compared to 9's complement.

1/2/9/10's complement ...

- Addition & subtraction using r's complement
  - ex: A – B (A : Minuend, B: Substrahend)
    - In both the cases below, bring both numbers in to same radix format.
      - Use same radix-1 complement.
- Examples
  - 20 – 35 using 1/2/9/10's complement with 10 bits representation
  - -133 – 97 using 1/2/9/10's complement with 10 bits representation
  - 55 + 97 using 1/2/9/10's complement with 10 bits representation
  - 97 – 55 using 1/2/9/10's complement with 10 bits representation

1's complement subtraction

- If both number are positive
  - no complement required.
  - Add both of them, keep carry as it is.
    - Carry will be used as magnitude, since we didn't complement operation.
- If we have used complement for either the subtrahend or minuend
  - If there is no carry in the final result
    - Check the MSB of the result
      - if it is '1', number is –ve, then do 1's complement, keep –ve sign
      - if it is '0', number is +ve, then don't do take complement, +ve sign
  - If there is carry
    - Add carry to the final result, rest same as above.
- If we have used complement for both the minuend and subtrahend
  - Result is always negative signed.
    - Take the 1's complement, keeping aside carry
    - then add carry, represent in negative.
  - Ex: -55 – 12

2's complement subtraction

- Same as 1's complement
  - Carry will be ignored, rest all things are same.

20-35

- using 1's complement
  - 0001_0100
  - 1101_1100 (-35 in 1's complement)
  - result
    - 1111_0000
    - MSB = 1 => Negative number
    - -15
- using 2's complement
  - Same as above
  - 0001_0100
  - 1101_1101 (-35 in 2's complement)
  - result
    - 1111_0001
    - MSB = 1 => Negative number
    - 0000_1110 = -(14+1) = -15

-133-97

- 1's complement
  - both are negative, result will be –ve
  - -133 : 10111_1010
  - -97 : 11001_110
  - add
    - 110001_1000
    - Carry: 1
    - rest complement: 01110_0111
    - -231 add carry = -231+1= -230
- 2's complement
  - both are negative, result will be –ve
  - -133 : 10111_1011 (2's complement)
  - -97 : 11001_111 (2's complement)
  - add
    - 110001_1010
    - Carry: 1 (ignore carry in 2's complement)
    - rest complement: 01110_0101 (1st complement)
      - 2's complement : 01110_0110

1's complement

- both numbers should have same number of bits.
- 1's complement, if carry is 1, then add it.
- 32-55
    - one of them is –ve
    - take 1's complement
        - 32 : 0100000
        - -55 : 1001000
        - Add both : 1101000
        - MSB is '1', it indicates –ve number
            - No need to do complement
            - -23
- -32-55
    - both are –ve
    - take 1's complement
        - -32 : 1011111
        - -55 : 1001000
        - add both:
            - 10100111
                - Carry is there, keep it aside
            - MSB is '1', indicates –ve number
            - Do the 1's complement then add carry
                - -88 + 1 = -87

1's complement

- -132-157
  - 10111_1011 (-132)
  - 10110_0010 (-157)
  - add both
    - 101101_1101
    - Since both we took complement, result is always -ve
    - do 1st complement
      - 10010_0010 (290)
      - -290
    - then add carry
      - -290 + 1 = -289

1's complement

- -132+157
  - 10111_1011 (-132)
  - 01001_1101 (157)
  - add both
    - 100001_1000
    - 24+1 = 25
- 35-37
  - 0100011 (35)
  - 1011010 (-37)
  - add both
    - 1111101, no carry
    - MSB is '1', it is a negative number
      - take complement : 0000010 = -2

Solutions

- +20 binary representation
  - 010100
- +35 binary " "
  - 0100011
- -35 in 1's complement method
  - 1011100
- Addition using 1's complement
  - 1110000
  - MSB is '1', so result is negative take 1's complement of sum
  - And represent negative sign in front of sum
  - Sum 1's complement =-0001111
  - 2's complement of -35 (1011101)
  - Addition=1011101+0010100 =1110001 result is negative
    -
- if there is any carry during 2's complement, ignore that
  - -35 => if 2's complement results in carry, then ignore that carry.

9's complement subtraction

- If both number are positive
  - no complement required.
  - Add both of them, keep carry as it is.
- If we have used complement for either the subtrahend or minuend
  - If there is no carry in the final result, then number is negative.
    - take the 9's complement of the number, keep –ve sign.
  - If there is carry, add carry to the number
- If complement is used for both the minuend and subtrahend
  - Result is always negative signed.
  - If there is a carry, add carry to the number.
    - take the 9's complement of the number, keep –ve sign.
  - Ex: -55 – 12
    - complement of both 44 + 87 = 131
      - add carry : 32
      - 9's complement with –ve sign : -67

20-35

- using 9's complement
  - 20
  - 64 (-35 in 9's complement)
  - result
    - 84
    - No carry, result in –ve
    - take 9's complement, keep –ve sign
      - -15
- using 10's complement
  - Same as above
  - 20
  - 66 (-35 in 10's complement)
  - result is –ve since no carry
    - 86
    - take 9's complement +1
    - -(14+1) = -15

10's complement subtraction

- 32-55
  - one of the numbers is –ve
  - take 10's complement
    - 32+45 = 77
    - 10's complement of 77
      - 9's complement + 1
      - 22+1 = 23
      - Since there is no carry in 77, result in –ve
      - -23 is final answer.

9's complement subtraction

- 32-55
  - one of the numbers is -ve
  -

9's complement

- -133 -97 using 9's complement
  - Convert both to complement since both are –ve
  - 866 + 902 = 1768
    - 9's complement, add carry to the number
    - 768 + 1 = 769
    - Take 9's complement of 769
      - 230
      - Since we did complement for both, result is negative.

Agenda

- Logic gates
- Truth tables
- Basic gates realization using universal gates
- D-morgan theorems and Boolean algebraic tricks for optimization
- pos to sop and sop to pos conversions
- K-map
- half adder and half subtractor and full adder and full subtractor
- Realization of half adder and half subtractor and full adder and full subtractor using nand and nor gates

Logic gates

- Buf
- Inv
- And
- Or
- NAND
- NOR
- Xor
  - If odd number of 1's are there, output is '1'
  - Boolean expression for 3 input XOR
    - Y = ~A~BC + A~B~C + ~AB~C + ABC = A^B^C
- Xnor
  - if even number of 1's are there, output is '1'
    - Y = ~ABC + A~BC + AB~C + ~A~B~C = A~^B~^C
- Tri state buffers

Logic gates ...

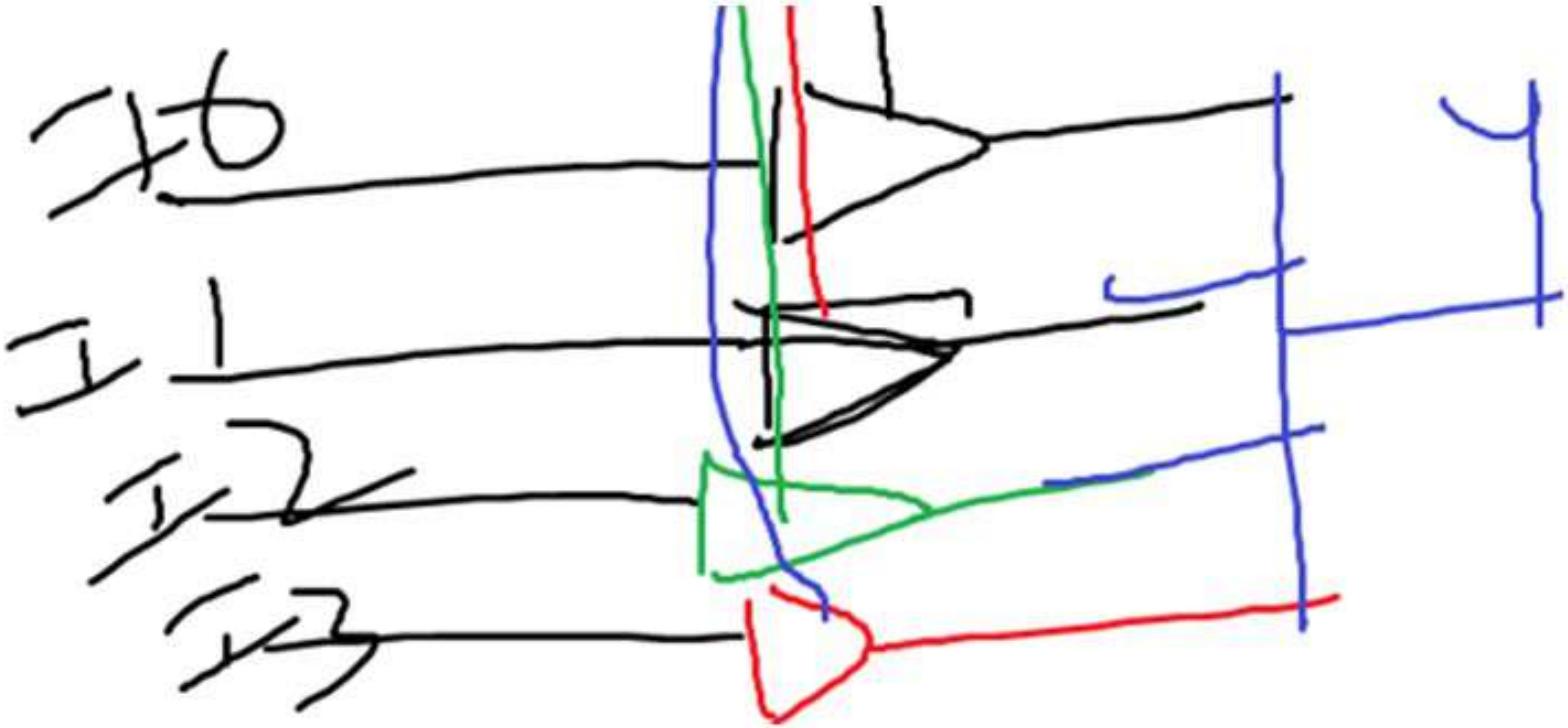
- Symbols
- Truth tables
- Universal gates
    - Why Nand is preferred over Nor?
        - cells are designed to have similar drive strength of pull up and pull down structures to have comparable rise and fall time.
        - NAND gate has better ratio of output high drive and output low drive as compared to NOR gate. Hence NAND gate is preferred over NOR.
- How logic gates are implemented in the chip?
- Realizing gates using universal gates
    - NAND
    - NOR

Tri state buffers

- has 2 inputs
  - A, Enable (active high)
  - If enable=1 => y = a
  - else y is floating, y = z
    - assign y = (en==1) ? a : z;
- There are also tri state buffers with active low enable
  - if enable=0 => y=a
  - if enable=1 => y = z (floating)
- Where are tri-state buffers used?
  - Used in GPIO's
    - General purpose IO
  - Used in Circuits where same port is driven by multiple components
    - We want only component to drive, other components should not drive. We keep tri state buffer between all these connections, which ensures that only one component will drive, rest of components will drive 'z'(tristate buffer will be open).
    - Diagram
      - among EN1 to EN4, only one will be high, rest all 0's, ensures that only one will be driving the output, rest will be driving 'z'

Tri state buffers

- I0, I1, I2, I3 are driving Y
  - But we want only one of them to take effect.
  - What happens if multiple of them drive Y at same time?
    - Leave the connection in floating state.
    - Floating state is represented in hardware using 'z' value.
      - At tri state buffer, if EN=0, output is not driven, it is 'z'



How logic gates are implemented in the chip

- Implemented using transistors based on technology node being targeted.
  - BJT
  - CMOS
    - Complementary MOS (1 PMOS and 1 NMOS)
  - FinFET
- PMOS
  - Gate=1 => Switch is open
  - Gate=0 => Switch is closed
- PMOS
  - Gate=0 => Switch is open
  - Gate=1 => Switch is closed
- NAND gate CMOS
  - PMOS : Parallel
  - NMOS : Series
- NAND truth table to correlate
  - 0 0 => 1
  - 0 1 => 1
  - 1 0 => 1
  - 1 1 => 0

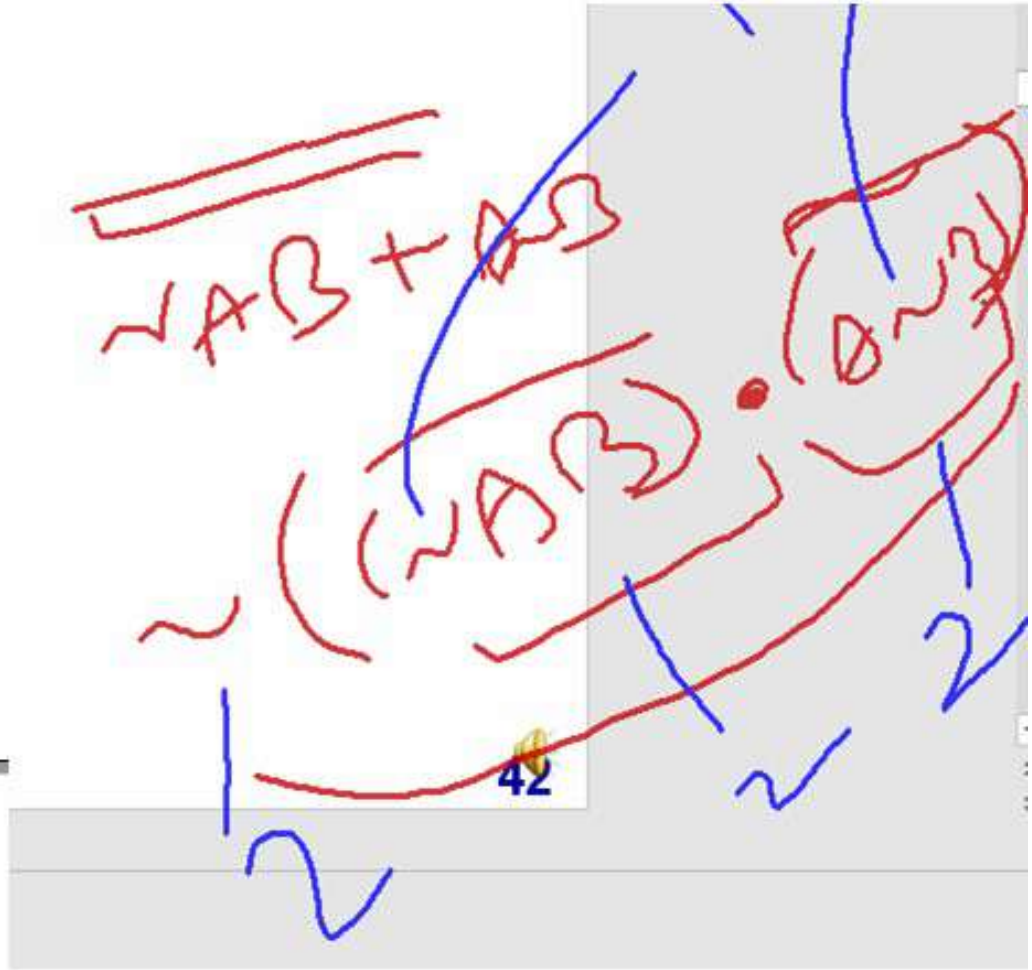
How logic gates are implemented in the chip ...

VLSIGURU

- NOR gates using CMOS
  - PMOS in series
  - NMOS in parallel
- NOR truth table to correlate
  - 0 0 => 1
  - 0 1 => 0
  - 1 0 => 0
  - 1 1 => 0
- OR gate using CMOS
  - NAND + Inverter = 3 CMOS
- AND gate also requires : 3 CMOS
- XOR gate
  - XOR = ~AB + A~B
    - ~A : 1 CMOS
    - ~B : 1 CMOS
    - ~AB : 3 CMOS
    - A~B : 3 CMOS
    - OR(+) : 3 CMOS
    - totally = 11 CMOS (un-optimized)
  - XOR = Bring whole expression in to NAND format (y = ~(a.b) = ~a OR ~b)

How logic gates are implemented in the chip ...

- XOR = Bring whole expression in to NAND format (y = ~(a.b) = ~a OR ~b)
  - ~AB + A~B
  - **~(~(**~AB + A~B**))**
  - ~(~(~AB).~(A~B))
    - ~(~AB) : Nand gate
    - ~(A~B) : Nand gate
    - ~(Nand1 . Nand2) : Nand gate
  - 8 CMOS required
- NAND: ~(A.B) = ~A+~B
- Homework
  - XNOR gates using CMOS
  - How many CMOS required?



Realizing gates using NAND universal gate

VLSIGURU

- To realize anything using NAND gates, bring the Boolean expression in to NAND format.
    - We may need to use ~~, which will bring expression to NAND format.
- AND
    - 2 NAND
    - 1 NAND + 1 NOT
- OR
    - (A + B)
    - bring in to NAND format
        - ~(~(A+B)) = ~(~A.~B) => NAND
        - ~A requires one NAND
        - ~B requires one NAND
        - whole expression requires one NAND
        - totally : 3 NAND gates required.
    - Using Nand and Not
        - 1 Nand and 2 Not
- Inverter
    - 1 Nand
- XOR
    - 4 NAND

Realizing gates using universal gates

- XNOR
  - unoptimized: 5 NAND
    - XNOR = ~(XOR) = 4+1(inverter) = 5
  - Optimized?
    - AB + ~A~B
    - bring above Boolean expression to NAND format ~(A.B)
    - ~(~(AB+~A~B))
    - ~(~(AB).~(~A~B))
      - ~(AB) : 1 Nand
      - ~(~A.~B) : 3 Nand
      - ~(2 expression and) : 1 Nand
      - totally 5 Nand gates
- Nor gate using NAND
  - NOR = ~(A+B) = ~A.~B = ~(~(~A.~B)) = 1+1+2 = 4

Realizing gates using NOR universal gate

VLSIGURU

- To realize anything using NAND gates, bring the Boolean expression in to NOR format.
    - We may need to use ~~, which will bring expression to NAND format.
    - NOR
        - ~(A+B)
        - ~A.~B
- NAND using NOR
    - NAND = ~(A.B) = ~A+~B = ~(~(~A+~B)) : 4 NOR gates.
- XOR using NOR
    - XOR = A~B + ~AB = (A~B + ~A).(A~B+B) = (~A+~B).(A+B)
    - = ~(~((~A+~B).(A+B))) = ~(~(~A+~B) + (~(A+B)) = 5 NOR
        - ~(~A+~B) : 1 NOR + 2 NOR for inversion
        - ~(A+B) : 1 NOR
        - ~(Expr1+Expr2) : 1 NOR
        - Totally 5 NOR gates required.
- Homework:
    - Do remaining
        - Inverter, XNOR, AND, OR using NOR

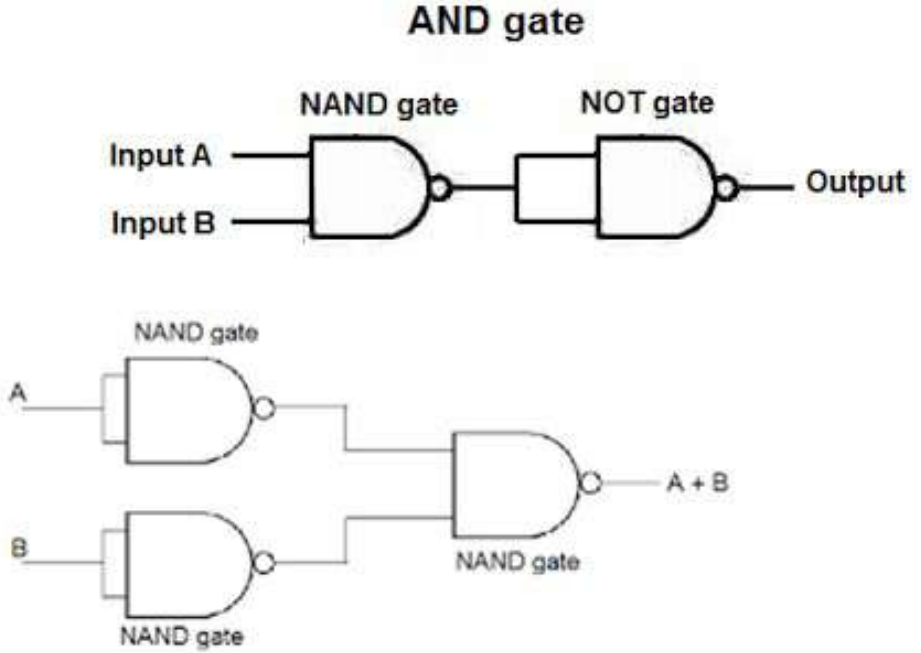
Realizing gates using NOR universal gate

VLSIGURU

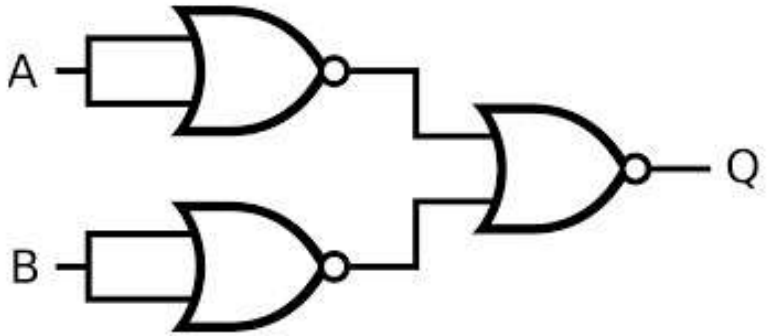
- NOR
  - ~(A+B)
  - ~A.~B
- XOR using NOR : Optimal method
  - XOR = ~(~(A~B + ~AB))
    - = ~((A+~B).(~A+B))
    - = ~(A+~B)+~(~A+B)
  - XOR = ~(AB + ~A~B)
    - = ~AB.~(~A.~B)
    - = ~(~(AB.~(~A.~B)))
    - = ~(~(AB) + ~(~A~B))
  - Optimal equation
    - ((A+(A+B)')' + (B+(A+B)')')' => XNOR
  - Write truth table, check all the outputs.
    - 0 0 1
    - 0 1 0
    - 1 0 0
    - 1 1 1
    - Above is behaving like a XNOR

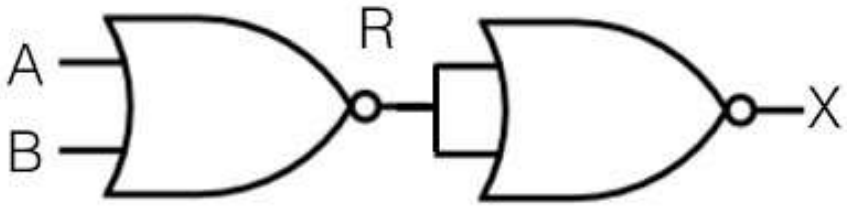
AB+BC+CA realization

- AB+BC+CA using Nand gates
  - Simplify
    - Various laws
    - K-maps
  - We know AND using NAND
    - Using this realize AB, BC, CA
  - Now realize OR gate using NAND

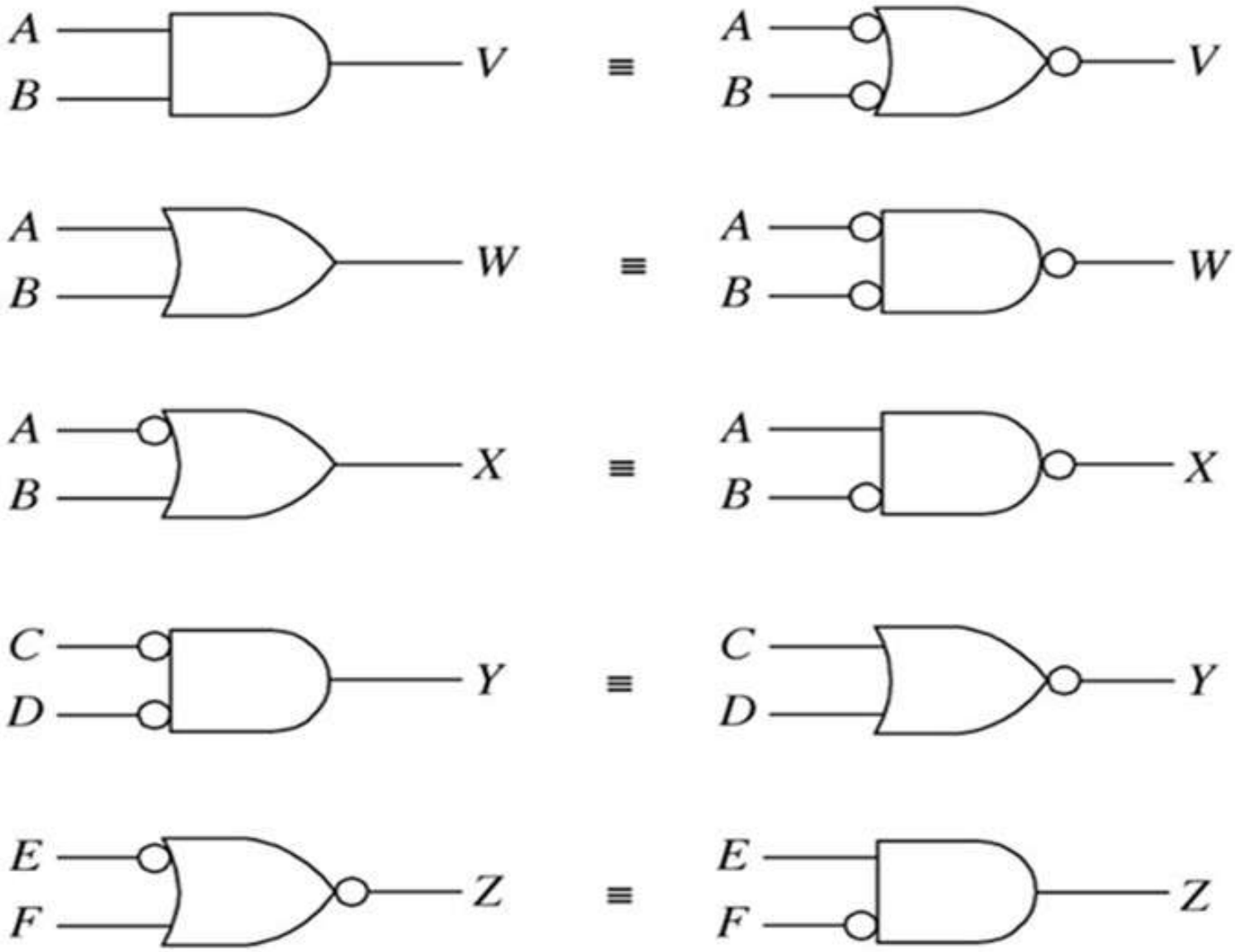


- AB+BC+CA using NOR gates
  - We know AND using NOR
    - Using this realize AB, BC, CA
  - Now realize OR gate using NOR





Rules for NAND and NOR



problems on NAND AND NOR

VLSIGURU

- implement y=AB+CD using NAND gates
- implement (~x+~y) (z+w)
- implement (~A+B)C+~F+DE using NAND and NOR gates
- A ~BC using 2 input NAND gates

Truth tables

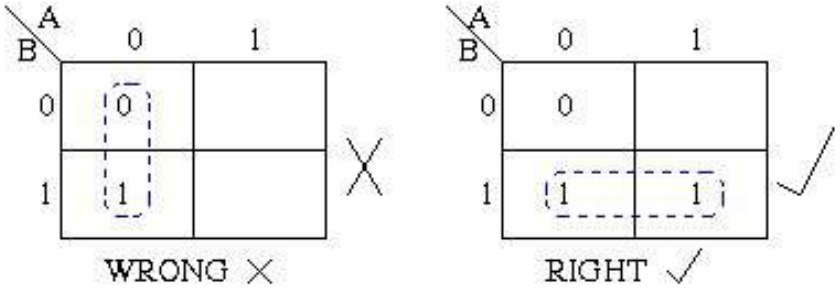
- Why truth tables are required?
  - to implement any logic using gates, we need to arrive with Boolean expression for all the design output
  - List down all design outputs
  - Write the truth table for outputs in terms of inputs
  - Truth table should be written for each output bit
- Ex:
  - FA truth table
  - A B C_i C_o S
  - 0 0 0 0 0
- Writing Boolean expressions
  - K-maps
    - Rules
      - numbers in a row or column should be in gray code pattern
      - Grouping should be done 1/2/4/8 (2 to the power)
      - Last row element can be grouped with 1st row element

K-maps rules

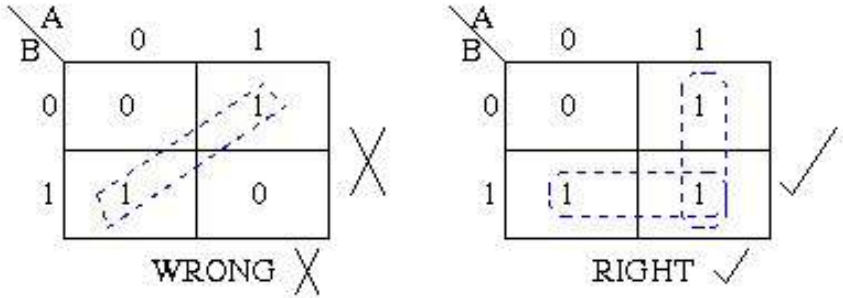
- number of boxes required in K-map = 2**inputs=2**3 = 8
- 8 can be done in : 2 rows, 4 column
- 8 can be done in : 4 row, 2 column
  - number_of_row * number_of_columns = total box count
- when we move from one column to the adjacent column, only one bit should change.
- when we move from one row to the adjacent row, only one bit should change.
  - same should be true for last row and 1st row
  - same should be true for last column and 1st column
- Groups may not include any cell containing a zero
- Groups may be horizontal or vertical, but not diagonal
- Groups must contain 1,2,4,8, or in general 2^n cells
- Each group should be as large as possible
- Each cell containing a one must be in at least one group.
- Groups may be overlapped.
- Groups may wrap around the table

K-MAP Rules

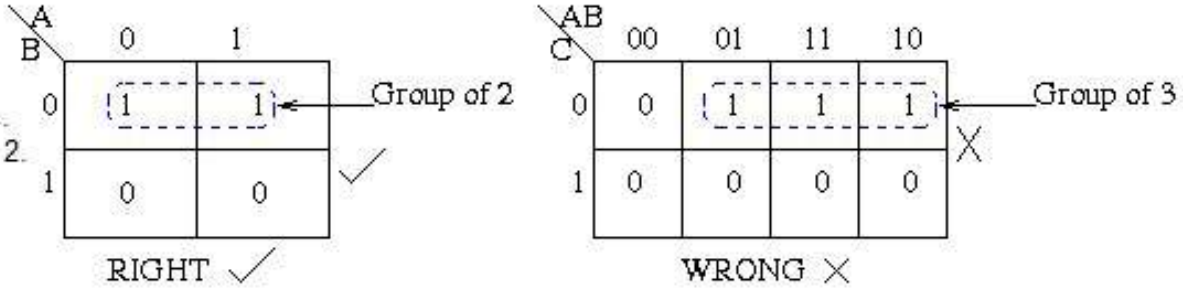
Groups may not include any cell containing a zero

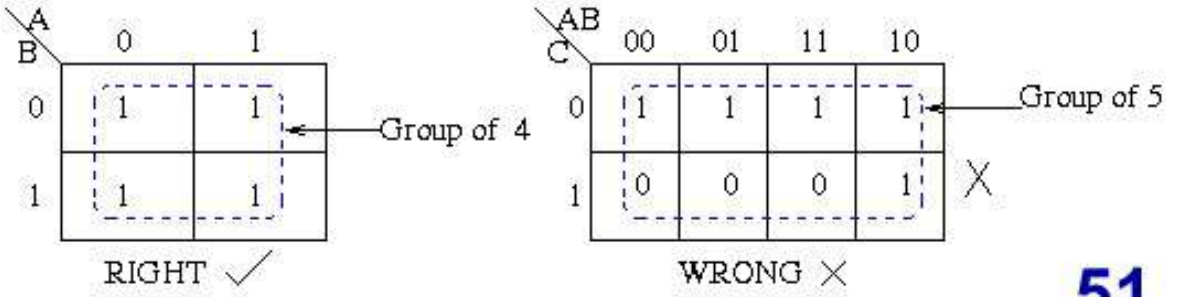


Groups may be horizontal or vertical, but not diagonal.

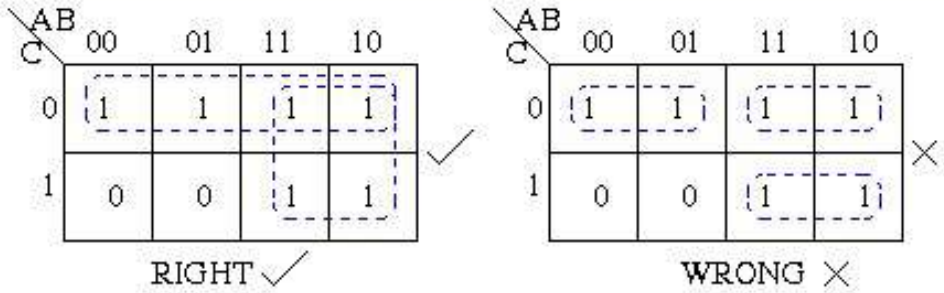


Groups must contain 1, 2, 4, 8, or in general 2n cells.
That is if n = 1, a group will contain two 1's since 21 = 2.
If n = 2, a group will contain four 1's since 22 = 4.

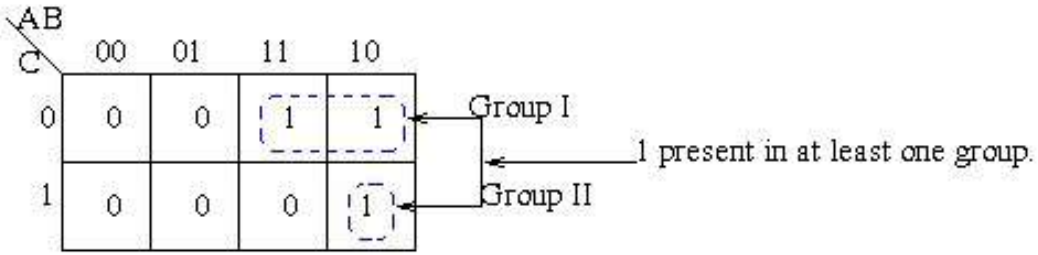




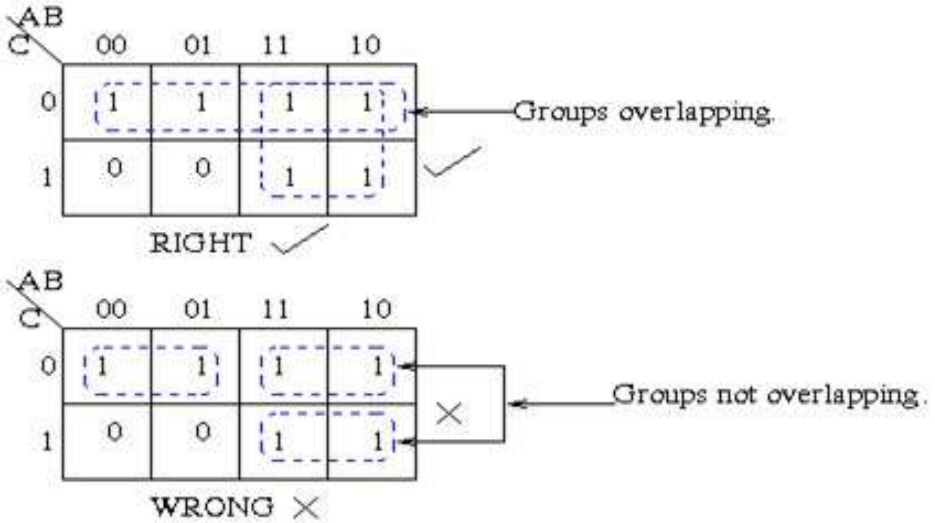
Each group should be as large as possible.



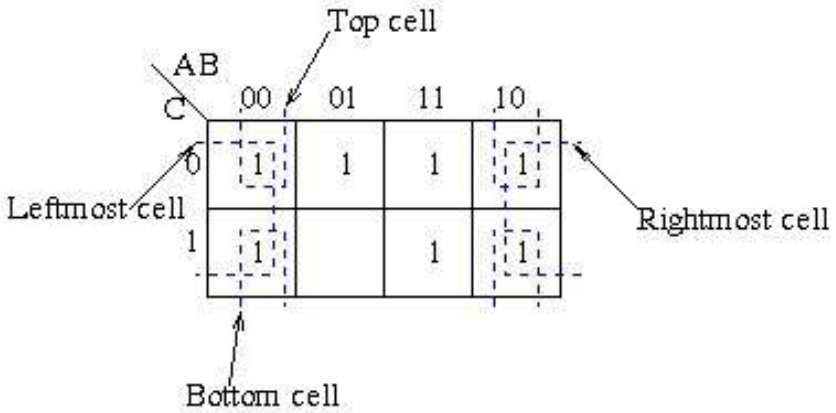
Each cell containing a one must be in at least one group.



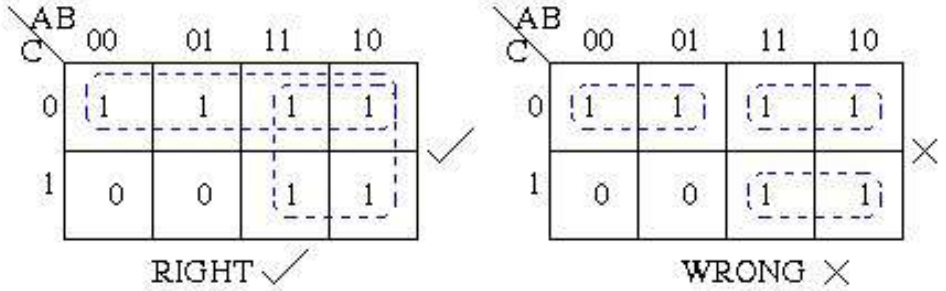
Groups may overlap.



Groups may wrap around the table. The leftmost cell in a row may be grouped with the rightmost cell and the top cell in a column may be grouped with the botto cell.



There should be as few groups as possible, as long as this does not contradict any of the previous rules.



Boolean expression simplification

VLSIGURU

Name	AND form	OR form
Identity law	$1A = A$	$0 + A = A$
Null law	$0A = 0$	$1 + A = 1$
Idempotent law	$AA = A$	$A + A = A$
Inverse law	$A\bar{A} = 0$	$A + \bar{A} = 1$
Commutative law	$AB = BA$	$A + B = B + A$
Associative law	$(AB)C = A(BC)$	$(A + B) + C = A + (B + C)$
Distributive law	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption law	$A(A + B) = A$	$A + AB = A$
De Morgan's law	$\overline{AB} = \bar{A} + \bar{B}$	$\overline{A + B} = \bar{A}\bar{B}$

Simplifying Boolean expression manually

Example 1

$$F = A.B.C + \bar{A} + A.\bar{B}.C$$

$$F = A.C.(B + \bar{B}) + \bar{A}$$

$$F = A.C.(1) + \bar{A}$$

$$F = \bar{\bar{A}}.C + \bar{A}$$

$$F = \bar{A} + C$$



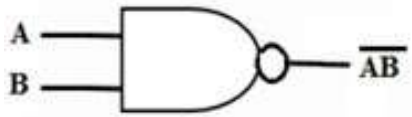
$A.A = A$	$A + A = A$	$A + \bar{A}B = A + B$
$A.\bar{A} = 0$	$A + \bar{A} = 1$	$\bar{A} + B = \bar{A}B$
$A.1 = A$	$A + 1 = 1$	$\bar{A}.B = \bar{A} + B$
$A.0 = 0$	$A + 0 = A$	

$$\begin{aligned}(A + B)(A + C) &= AA + AC + AB + BC \\ &= A + AC + AB + BC \\ &= A(1 + C + B) + BC \\ &= A \cdot 1 + BC \\ &= A + BC\end{aligned}$$

DeMorgan's Theorem

$$\overline{A.B} = \overline{A}+\overline{B}.$$

$$\overline{A+B} = \overline{A}.\overline{B}$$









For a 3-variable input

$$\overline{A.B.C} = \overline{A}+\overline{B}+\overline{C}$$

and also

$$\overline{A+B+C} = \overline{A}.\overline{B}.\overline{C}$$

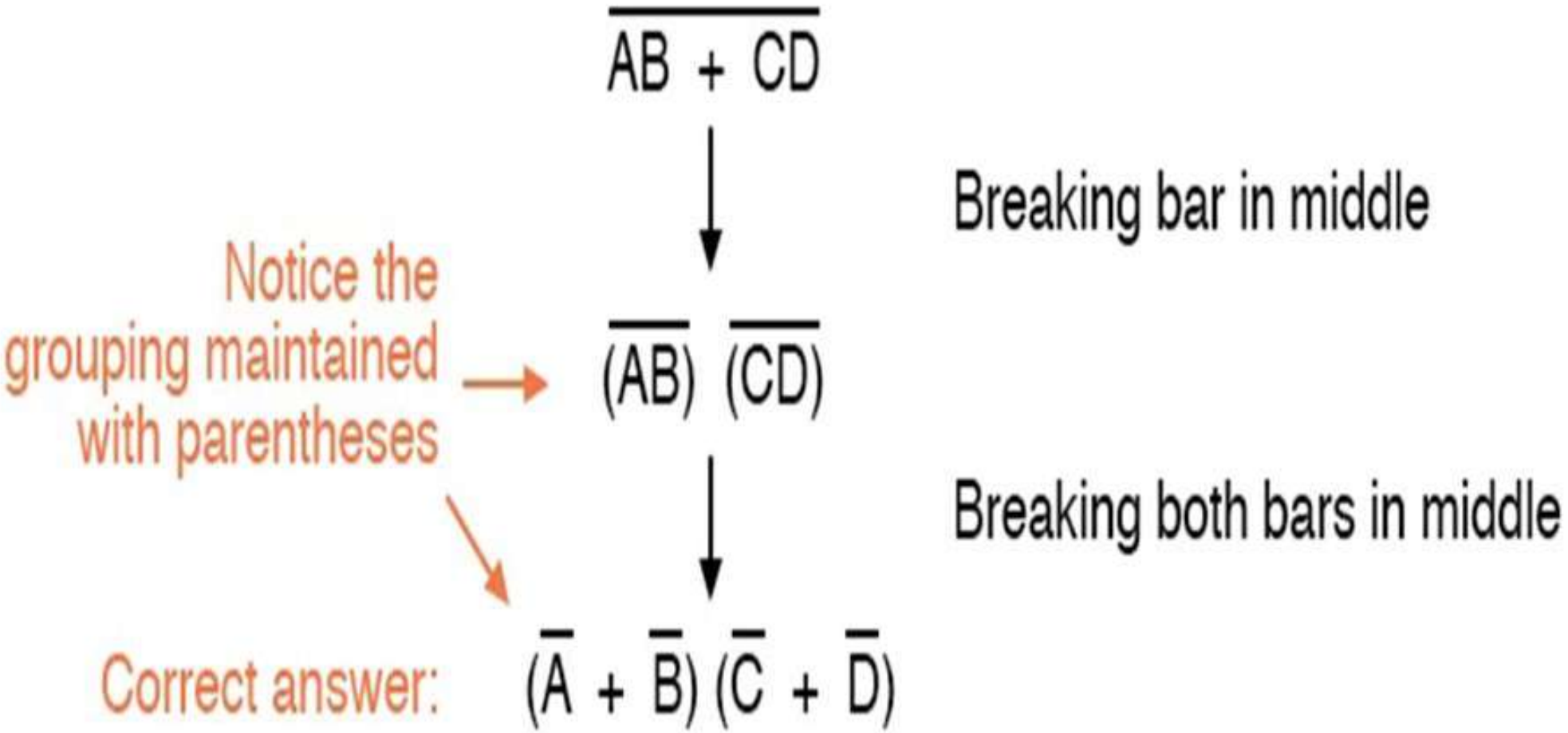
For a 4-variable input

$$\overline{A.B.C.D} = \overline{A}+\overline{B}+\overline{C}+\overline{D}$$

and also

$$\overline{A+B+C+D} = \overline{A}.\overline{B}.\overline{C}.\overline{D}$$

DeMorgan's Theorem ...



Boolean expression simplification

VLSIGURU

- A + 1 = 1
- A + A = A
- A + ~A = 1
- A + 0 = A
- A * 1 = A
- A * A = A
- A * A bar = 0
- A * 0 = 0
- ~(~A) = A
- A + B + 1 = 1
- (A + B) (A + C) = A+BC
- A + ~A * B = A + B
  - Using distributive law

Boolean expression simplification

$$B + A\acute{B}C$$

$$B + \acute{A} + B\acute{C}$$

DeMorgan's law

$$\acute{A} + \acute{B} + \acute{C}$$

DeMorgan's law

$$B + \acute{B} + \acute{A} + \acute{C}$$

Commutative law

$$\left(B + \acute{B}\right) + \acute{A} + \acute{C}$$

Associative law

$$1 + \acute{A} + \acute{C}$$

Inverse law

$$1$$

True law

Comparing two Boolean expressions

- If both Boolean expression have same truth table for all the input combinations, then both Boolean expressions are same.
  - First list down how many variables are there in both expressions
    - Simplify both the Boolean expressions
    - If number variables in both sides don't match, they are different.
  - After that we try truth table

(i) *When* $A = 0, \quad B = 0,$

$$\overline{A \cdot B} = \overline{0 \cdot 0} = \bar{0} = 1$$

and $\bar{A} + \bar{B} = \bar{0} + \bar{0} = 1 + 1 = 1$

Hence $\overline{A \cdot B} = \bar{A} + \bar{B}$

(ii) *When* $A = 0, \quad B = 1,$

$$\overline{A \cdot B} = \overline{0 \cdot 1} = \bar{0} = 1$$

and $\bar{A} + \bar{B} = \bar{0} + \bar{1} = 1 + 0 = 1$

Hence $\overline{A \cdot B} = \bar{A} + \bar{B}$

(iii) *When* $A = 1, \quad B = 0,$

$$\overline{A \cdot B} = \overline{1 \cdot 0} = \bar{0} = 1$$

and $\bar{A} + \bar{B} = \bar{1} + \bar{0} = 0 + 1 = 1$

Hence $\overline{A \cdot B} = \bar{A} + \bar{B}$

(iv) *When* $A = 1, \quad B = 1,$

$$\overline{A \cdot B} = \overline{1 \cdot 1} = \bar{1} = 0$$

and $\bar{A} + \bar{B} = \bar{1} + \bar{1} = 0 + 0 = 0$

Hence $\overline{A \cdot B} = \bar{A} + \bar{B}$

problems

- Realization of full subtractor using nand and nor gates
- F(A,B,C)=(A.B.C) + (A'.B')+ (A.B'.C) + (A.C')
  - how to convert this SOP to POS?
- AB+ A~C+BC = AB+A~C optimize the logic

One style of simplification

VLSIGURU

- AB+ A~C+BC = AB+A~C optimize the logic
- Bring all minterms in to 3 variable format.
- Then regroup all the variables
- Take the common expression in such a way
  - Expression should left as (A+~A) kind of format.
- ABC + AB~C+ A~CB + A~C~B + BCA + BC~A
- ABC + AB~C+ AB~C + A~B~C + ~ABC
- ABC + AB~C + A~B~C + ~ABC
- ABC + ~ABC + AB~C + A~B~C
- BC(A+~A) + A~C(B+~B)
- BC + A~C

- AB+ A~C+BC = AB+A~C optimize the logic

- Reduction using K-maps
  - 3 variables: A, B, C
  - 4 columns * 2 rows K-map
    - Fill the 1's based on min terms

SOP and POS representations

VLSIGURU

- Canonical Form
  - In Boolean algebra, Boolean function can be expressed as
  - Canonical Disjunctive Normal Form known as **minterm**
  - and some are expressed as
  - Canonical Conjunctive Normal Form known as **maxterm**.
- In **Minterm**, we look for the functions where the output results in "1" while in **Maxterm** we look for function where the output results in "0".
- We perform **Sum of min terms** also known as **Sum of products (SOP)**.
- We perform **Product of Maxterms** also known as **Product of sum (POS)**.
  Boolean functions expressed as a sum of min terms or product of maxterms are said to be in **canonical form**.
- Standard Form – A Boolean variable can be expressed in either true form or complemented form. In standard form Boolean function will contain all the variables in either true form or complemented form while in canonical number of variables depends on the output of SOP or POS.
- A Boolean function can be expressed algebraically from a given truth table by forming a :
  - minterm for each combination of the variables that produces a 1 in the function and then taking the OR of all those terms.
  - maxterm for each combination of the variables that produces a 0 in the function and then taking the AND of all those terms.
- Min terms using Sigma
  - Sum of product format
- Max terms using Pie
  - Pie gives product of sum

SOP and POS representations

VLSIGURU

Variables			Min terms	Max terms
A	B	C	m_i	M_i
0	0	0	A' B' C' = m 0	A + B + C = M 0
0	0	1	A' B' C = m 1	A + B + C' = M 1
0	1	0	A' B C' = m 2	A + B' + C = M 2
0	1	1	A' B C = m 3	A + B' + C' = M 3
1	0	0	A B' C' = m 4	A' + B + C = M 4
1	0	1	A B' C = m 5	A' + B + C' = M 5
1	1	0	A B C' = m 6	A' + B' + C = M 6
1	1	1	A B C = m 7	A' + B' + C' = M 7

SOP implementation

- Example – Express the Boolean function F = A + B'C as standard sum of min terms.

- Solution –
- A = A(B + B') = AB + AB'
- This function is still missing one variable, so
- A = AB(C + C') + AB'(C + C') = ABC + ABC'+ AB'C + AB'C'
- The second term B'C is missing one variable; hence,
- B'C = B'C(A + A') = AB'C + A'B'C
- Combining all terms, we have
- F = A + B'C = ABC + ABC' + AB'C + AB'C' + AB'C + A'B'C
- But AB'C appears twice, and
- according to theorem 1 (x + x = x), it is possible to remove one of those occurrences. Rearranging the min terms in ascending order, we finally obtain
- F = A'B'C + AB'C' + AB'C + ABC' + ABC
  - m1 + m4 + m5 + m6 + m7
- SOP is represented as Sigma(1, 4, 5, 6, 7)

POS implementation

- Example – Express the Boolean function F = xy + x'z as a product of maxterms
- Hint: Distributive law, Inverse law

- Solution –
- F = xy + x'z
- = (xy + x')(xy + z)
- = (x + x')(y + x')(x + z)(y + z)
- = (x' + y)(x + z)(y + z)
- x' + y = x' + y + zz'
- = (x'+ y + z)(x' + y + z') (x + z)
- = x + z + yy'
- = (x + y + z)(x + y' + z) y + z
- = y + z + xx'
- = (x + y + z)(x' + y + z)
- F = (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z')
- = M0*M2*M4*M5
- POS is represented as Pi(0, 2, 4, 5)
- Easiest way
  - Take SOP, then take remaining numbers => POS

POS, SOP realization using Mux

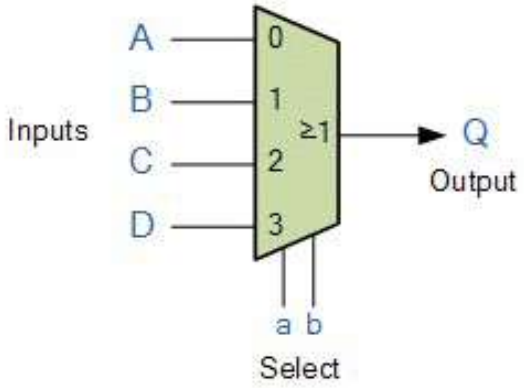
- Q23
  - Minterms
  - D used as input variable
    - 2 4X1 Mux and 1 2X1 mux
- Q24
  - Maxterms

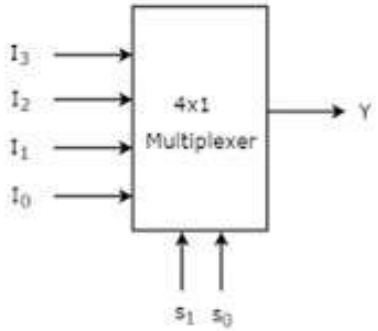
Combinational circuits

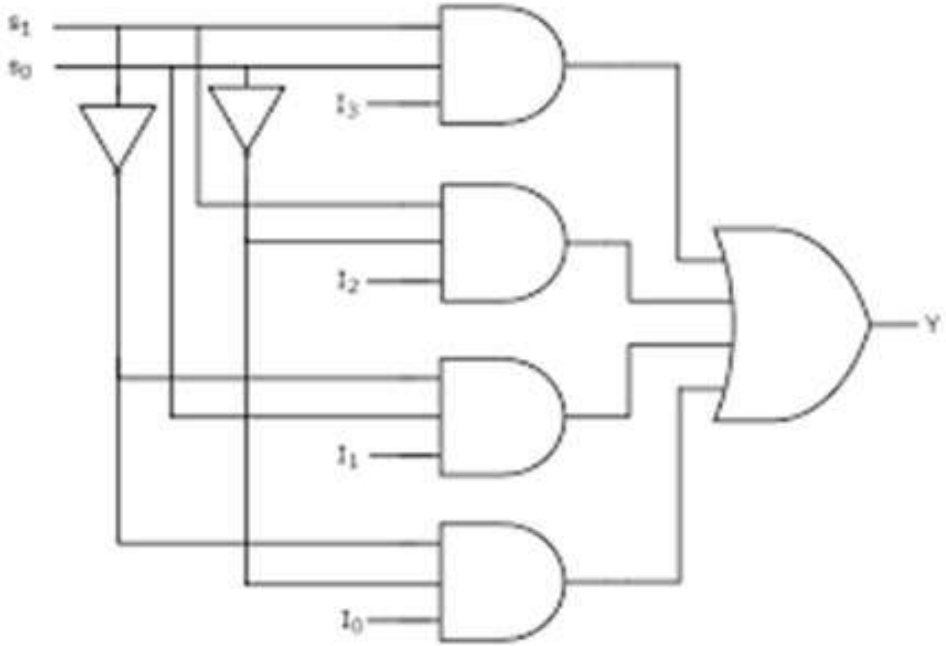
- Multiplexor
- De-Multiplexor
- Encoder
- Decoder
- Priority encoder
- Comparator
- Half adder, Full adder
- Half subtractor, Full subtractor

Multiplexer

- Used to select one among multiple inputs.
- Generally used: 2x1, 4x1, 8x1, 16x1
- Other types also possible: 3x1



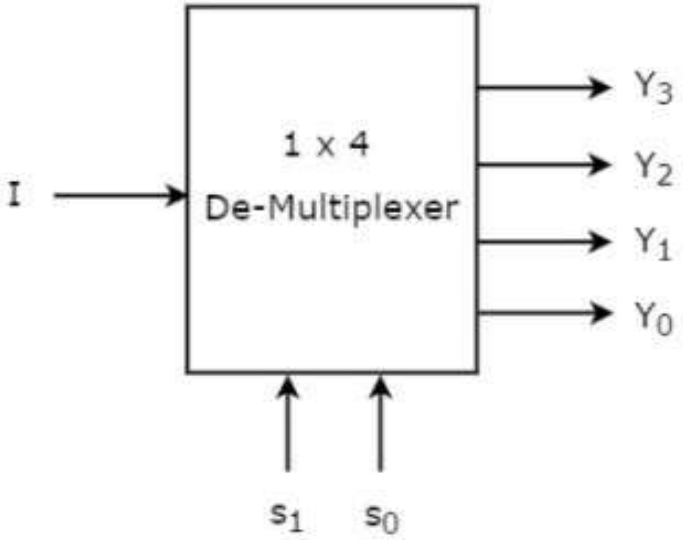


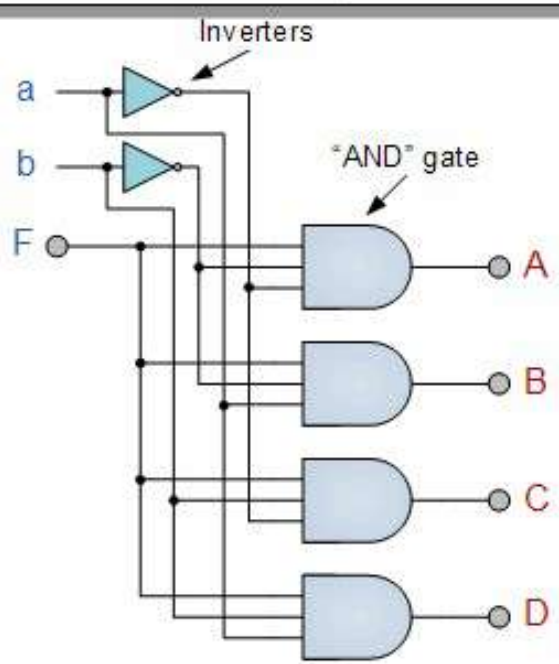


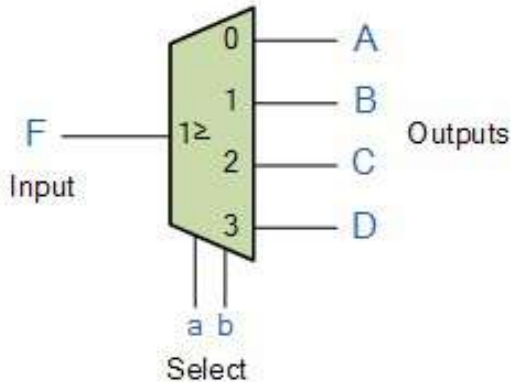
Selection Lines		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

Demultiplexer

- Used to connect a single source to multiple destinations.
  - ex: Drive same signal to various outputs based on the selection.







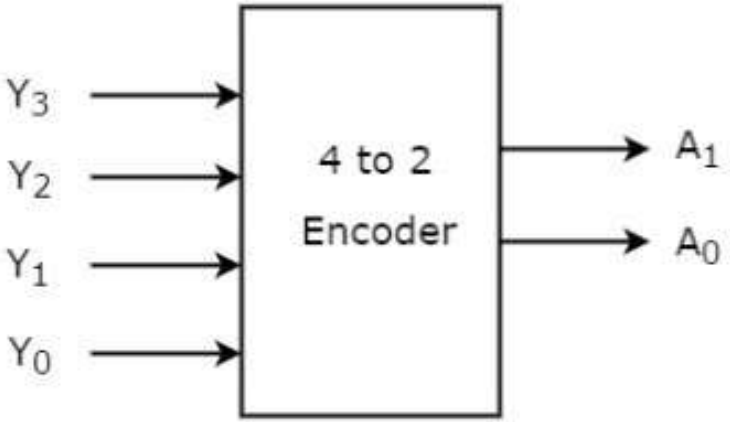
Input	Select Lines	Output Lines
I	S_1 S_0	D_0 D_1 D_2 D_3
I	0 0	1 0 0 0
I	0 1	0 1 0 0
I	1 0	0 0 1 0
I	1 1	0 0 0 1

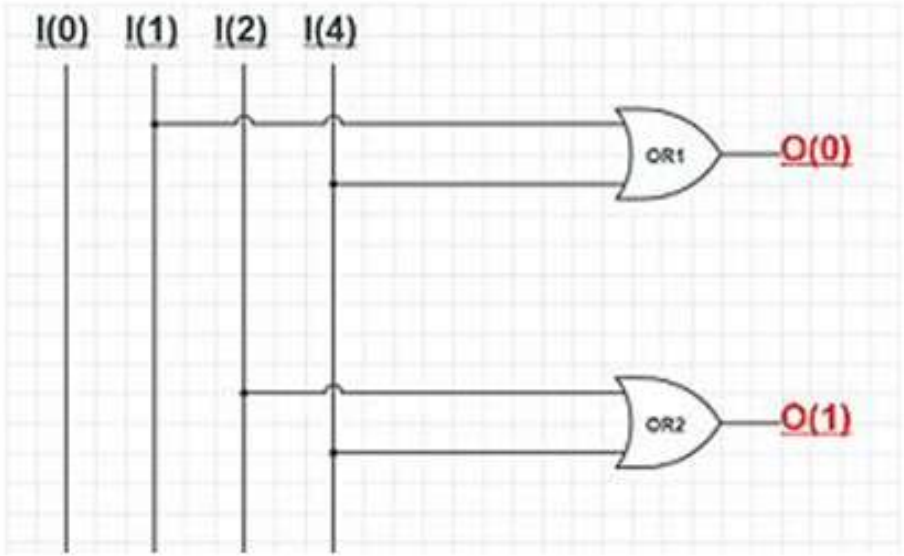
Mux, Demux

- Mux
  - It is combinational circuit that selects binary information from one of many input lines and direct it to o/p line
  - it is simply a Data selector
- Demux
  - one input many outputs
  - data distributor
  - reverse operation of mux

Encoder

- Applications: Decimal to binary encoding in calculators.
  - Requires only one bit of the all inputs to be '1'
  - If this violates output is 0's





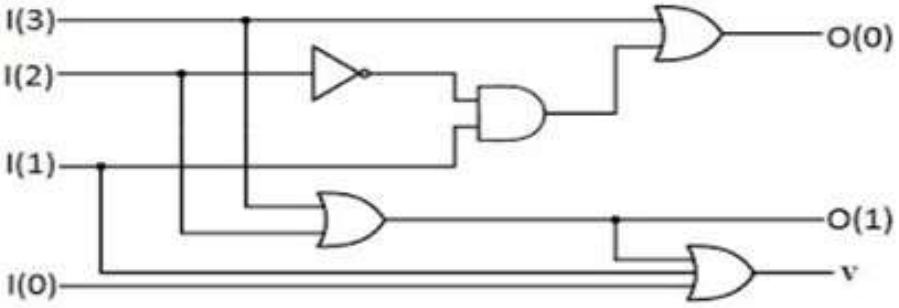
INPUTS				OUTPUTS	
Y3	Y2	Y1	Y0	A1	A0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

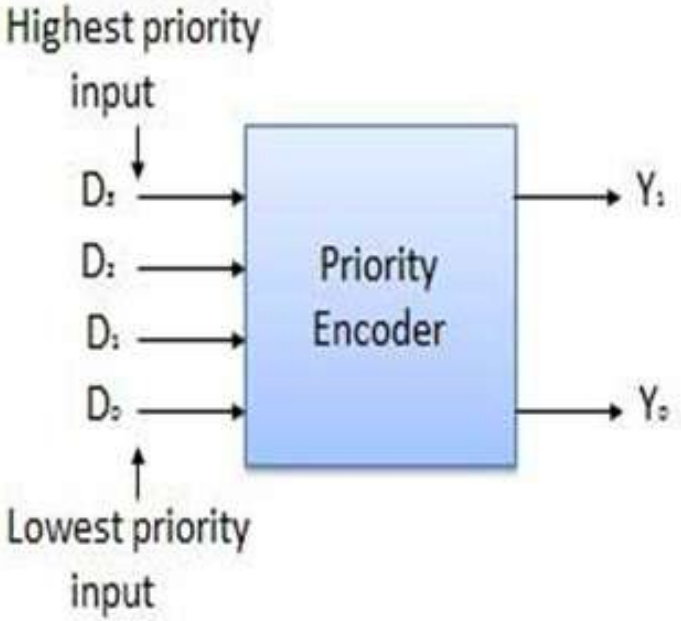
Priority Encoder

VLSIGURU

- Applications: Decimal to binary encoding in calculators.
  - special type of encoder whose output corresponds to the currently active input which has the highest priority.so when an input with a higher priority is present, all other inputs with a lower priority will be ignored.
  - x in truth table indicates that those elements are don't care
    - They can be '1' or '0', output will still be same.
    - output don't have any dependency on those variables.

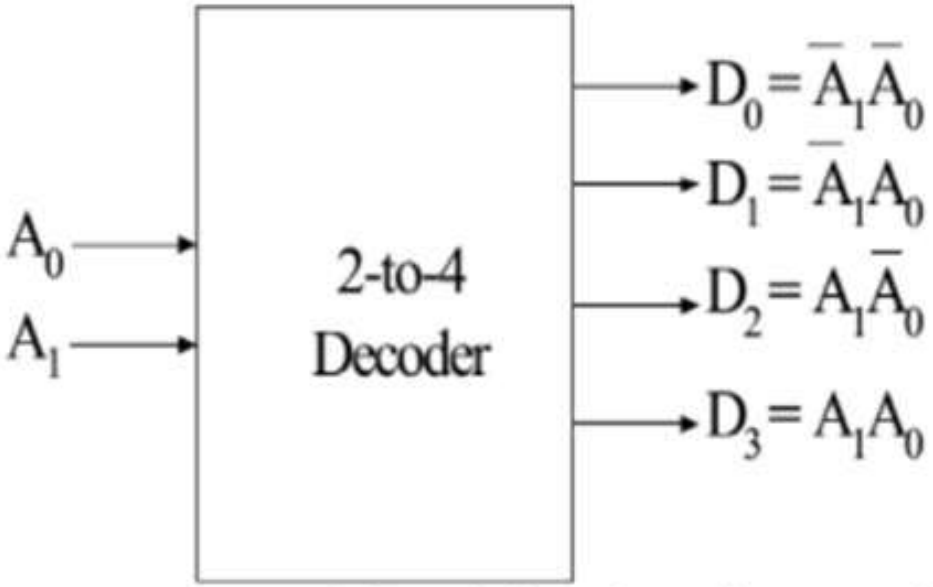
Inputs				Outputs		
D_0	D_1	D_2	D_3	Y_1	Y_0	V
0	0	0	0	×	×	0
1	0	0	0	0	0	1
×	1	0	0	0	1	1
×	×	1	0	1	0	1
×	×	×	1	1	1	1



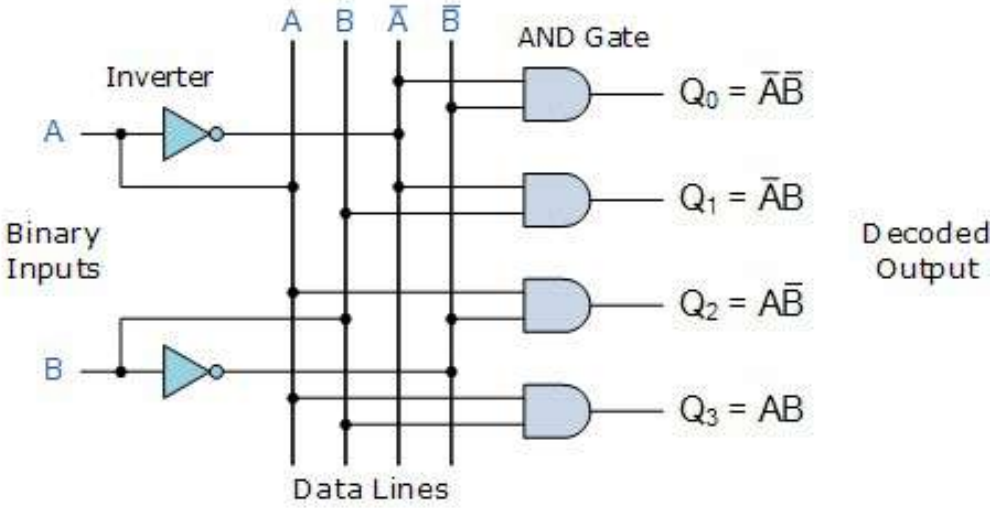


Decoder

- Simple usage example: Traffic light

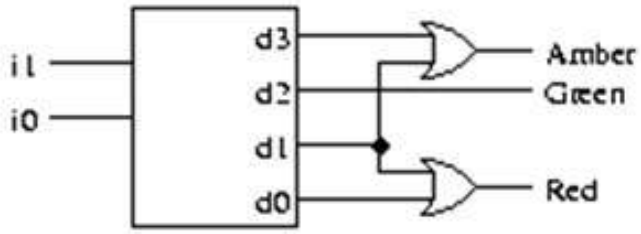


A 2-to-4 decoder without enable



Decimal #	Input		Output			
	A_1	A_0	D_0	D_1	D_2	D_3
0	0	0	1	0	0	0
1	0	1	0	1	0	0
2	1	0	0	0	1	0
3	1	1	0	0	0	1

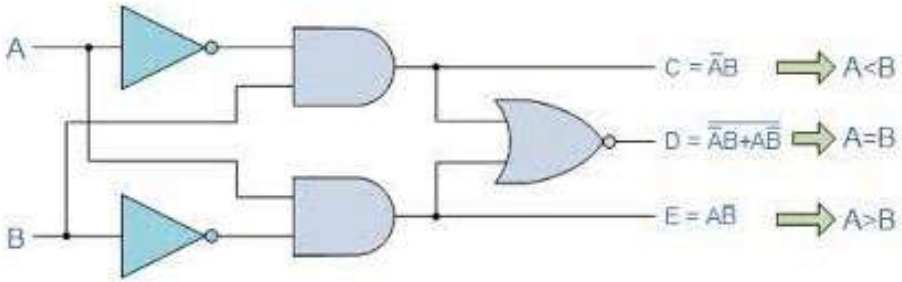
Truth table for 2-to-4 decoder



Comparator

•





a	b	a=b	a>b	a<b
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

a=b: a'b' + ab

a>b: ab'

a<b: a'b

The logic for a=b is also: (a'b + ab')'

2 bit Comparator

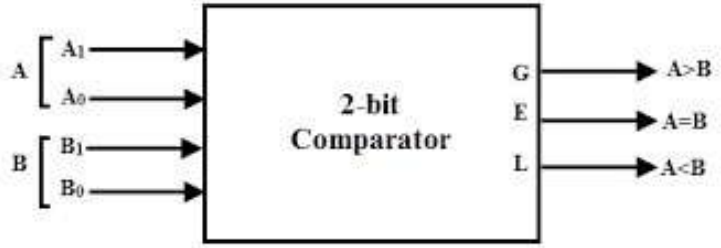
VLSIGURU

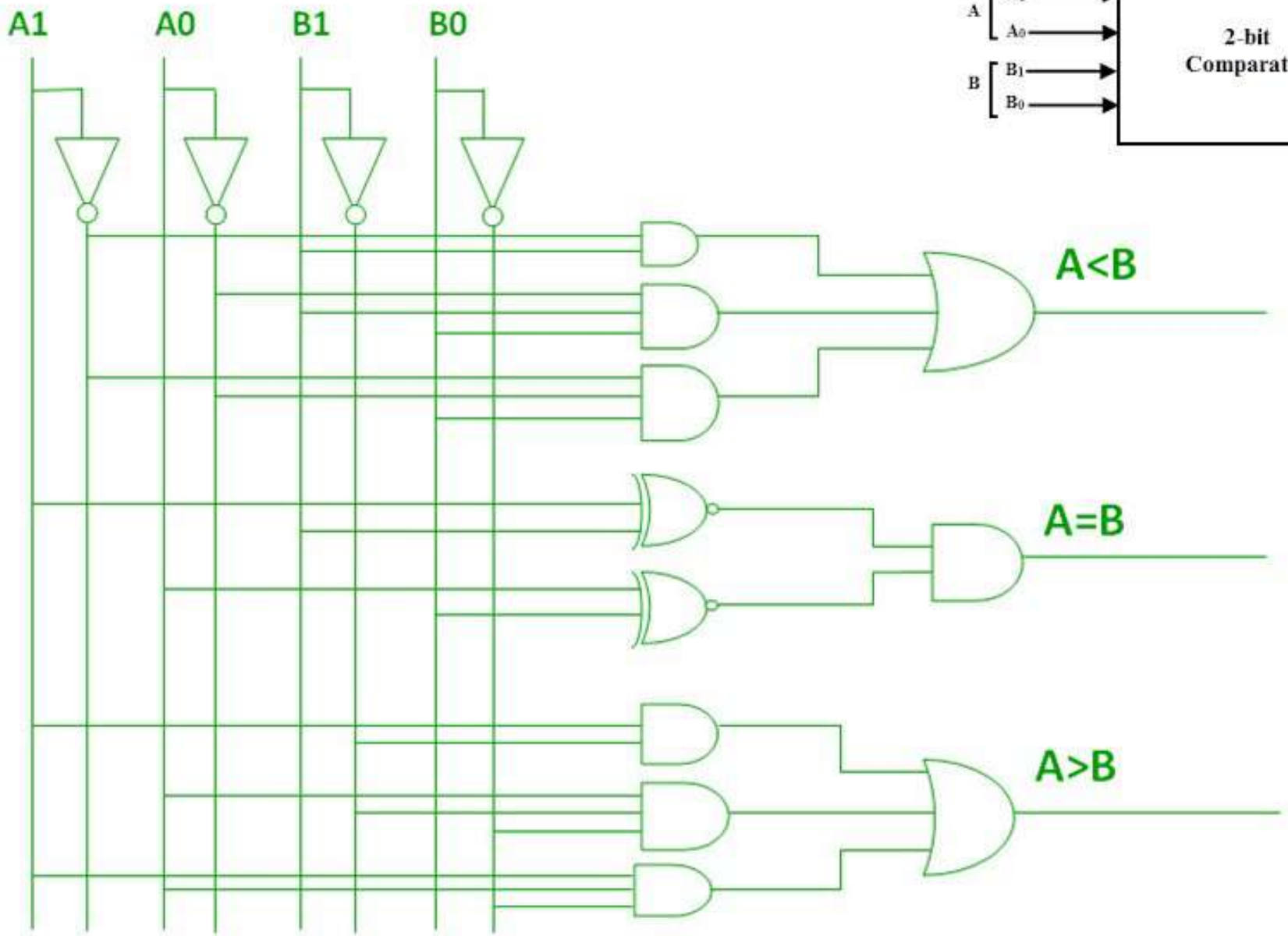
•

Inputs				Outputs		
A_1	A_0	B_1	B_0	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

2 bit Comparator ...

•



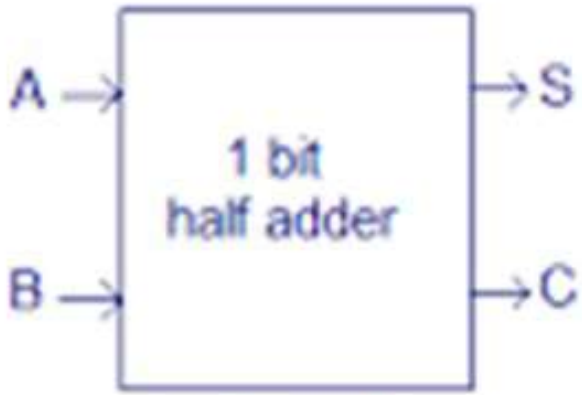


Half adder

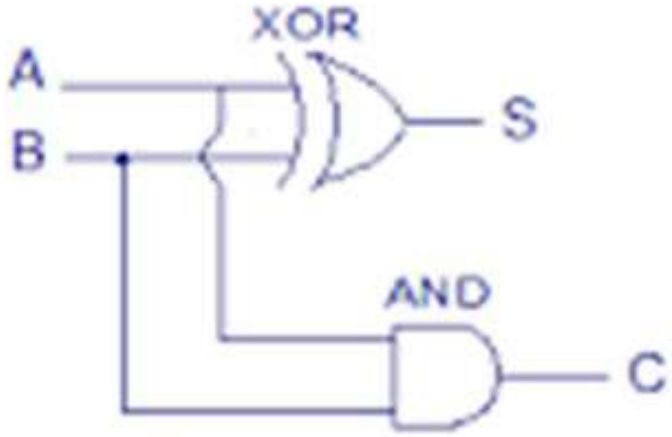
- no carry, only 2 inputs

Inputs		Outputs	
A	B	S	C
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

Truth table



Schematic

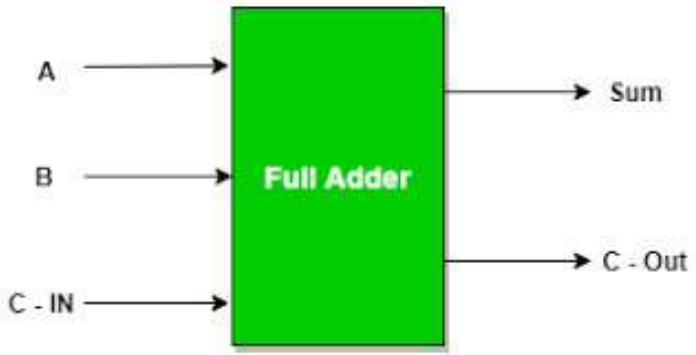


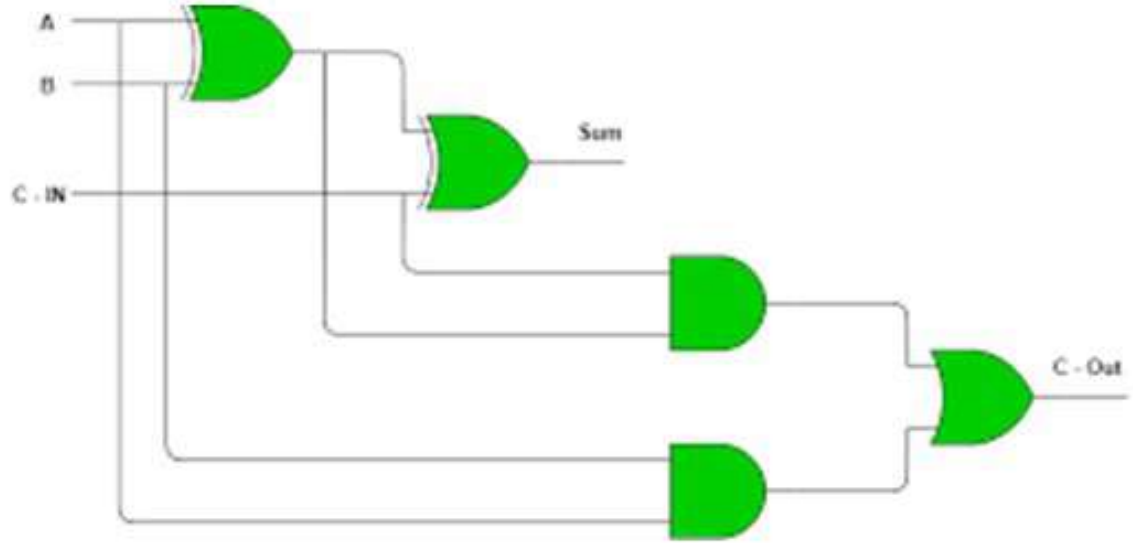
Realization

Full adder

VLSIGURU

•





Inputs			Outputs	
A	B	C – IN	Sum	C - Out
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Full adder using half adder

VLSIGURU

- Can we arrive at below diagram using FA and HA truth table?

